

Plan: Comercios Asociados + Flujo de sellos con boleta

Club Patio Curauma

1. Concepto

Hoy todos los comercios viven en `entrepreneur_profiles` sin distinción. Se introduce un segundo tipo de comercio con identidad, presentación y **flujo de sellos propio**:

	Emprendedor (actual)	Comercio Asociado (nuevo)
Quién es	Artesano / persona (joyería, deco, artesanía)	Local/marca establecida (cafetería, restaurante, gimnasio)
Flujo de sellos	El vendedor escanea al cliente y confirma en caja	Auto-servicio : el cliente escanea el QR del local, ingresa el monto y sube foto de la boleta
Aprobación	Inmediata por el vendedor	Automática, con auditoría posterior del moderador (revisa boletas, anula las falsas)
Cálculo de sellos	<code>calcularSellos(monto)</code> (1–4 según monto)	El mismo <code>calcularSellos(monto)</code> — la mecánica no cambia
Meta de premio	5 sellos	5 sellos (igual)

La diferencia funcional real es **cómo se gana el sello** (auto-servicio con prueba) y la **auditoría de boletas**.

2. Modelo de datos

2.1 entrepreneur_profiles — campo nuevo

- `tipo: "emprendedor" | "asociado"` — default `"emprendedor"` (lectura tolerante: si falta, es emprendedor). Nada existente se rompe.

2.2 system_logs — campos nuevos en sellos por boleta

- `metodo: "CLIENT_BOLETA"` (distinto de `VENDOR_SCAN`)
- `boletaUrl` — URL de la imagen en Storage · `boletaPath` — ruta interna (para borrarla en limpieza)
- `monto`, `numSellos`, `tipo: "FIDELIZACION"` (como hoy) · `revisada?` — marca de auditoría

2.3 Firebase Storage — nueva carpeta

- `boletas/{vendorId}/{timestamp}_{uid}.jpg` — separada de `entrepreneur_photos/` para poder limpiarla sin tocar logos.

2.4 Reglas de seguridad

- Usuario autenticado puede **subir** a `boletas/`. Lectura restringida a roles staff (moderador/admin/director).

3. Fundación compartida

Nuevo: `src/lib/tipoComercio.ts` — el tipo, etiquetas ("Comercio Asociado" / "Emprendedor"), estilos de badge, y lector `getTipoComercio(data)` (default `"emprendedor"`) + helper `esAsociado(data)`. Punto único de verdad para no repetir strings por toda la app.

4. Fases de implementación

FASE 1 — Fundación + carga masiva (sin UI visible para el socio)

1. Crear `src/lib/tipoComercio.ts`.
2. Agregar campo `tipo` a `comercios_a_cargar.mjs` y al script `_ingresar_comercios.mjs`.
3. Default propuesto para los 6: **Asociados:** Fika, El Refugio, Máster Brod, Pomarus, Moviclean · **Emprendedor:** Antojitos de la Nutri (*ajustable*).

Entregable: los 6 comercios cargados con su `tipo` correcto. Aún sin cambios visuales.

FASE 2 — Panel Moderador: cambiar el tipo

- Toggle **Emprendedor** ⇌ **Comercio Asociado** que escribe `tipo` en `entrepreneur_profiles/{uid}`.
- Cumple asignar el tipo "en ambos" (carga masiva + panel). Archivos: `src/app/moderador/page.tsx` y/o `src/app/directorio/page.tsx`.

FASE 3 — Superficies visibles para el socio

- **Directorio / Descubre:** sección separada "Comercios Asociados" (arriba) y "Emprendedores" (abajo); **badge** "Asociado" en la tarjeta; **filtro** Asociados / Emprendedores. Archivos: `src/app/directorio/page.tsx`, `src/app/descubre/page.tsx`.
- **Home / Destacados:** los Asociados aparecen **priorizados**.

FASE 4 — Flujo de sellos por boleta (auto-servicio) — núcleo

Disparador: el cliente escanea el QR del local (`/canje?localId=<uid>`), que **ya existe**.

1. **Detección de tipo en /canje:** si `tipo !== "asociado"` → flujo actual sin cambios. Si `tipo === "asociado"` → nuevo formulario: (1) monto de la compra (\$1–\$150.000), (2) foto de boleta obligatoria, (3) botón "Obtener mis sellos".
2. **Subida de imagen** a `boletas/{localId}/{timestamp}_{uid}.jpg` (mismo patrón `uploadBytes` + `getDownloadURL` de los logos).
3. **Nueva API** `POST /api/handshake/boleta-scan`: auth + rate limit; valida monto y que el local sea `tipo: "asociado"`; `calcularSellos(monto)`; otorga sellos (mismos campos); registra en `system_logs` con `metodo: "CLIENT_BOLETA"`, `boletaUrl`, `boletaPath`; push al cliente (reutiliza `vendor-scan`).
4. **Confirmación:** misma celebración de sello del flujo actual.

Anti-fraude (auto-reportado): boleta obligatoria · rate limit por IP + cooldown por usuario+local (reutiliza `lastVendorScans`) · tope de monto (\$150.000) · todo queda con boleta para que el moderador anule las falsas · *opcional:* dejar el sello "pendiente de revisión" si se prefiere control previo (decisión \$5).

FASE 5 — Auditoría de boletas en Moderador

Sobre `src/app/moderador/sellos/page.tsx` (que ya lista sellos y permite anular):

1. **Ver la boleta:** miniatura de `boletaUrl` en los registros `CLIENT_BOLETA` (click → imagen completa).

2. **Anular sello falso:** ya existe `POST /api/admin/anular-sello` (resta sello, actualiza Google Wallet, marca anulada). **Extender** para que al anular **borre la imagen** de Storage.
3. **Limpiar imágenes:** acción nueva para borrar en lote boletas antiguas de Storage conservando el registro en `system_logs`. Nueva API `POST /api/admin/limpiar-boletas` (solo staff).

5. Decisiones abiertas (requieren tu confirmación)

1. **¿Aprobación previa o posterior?** El requisito dice "se le dan los sellos" al subir la boleta → **otorgar de inmediato** y auditar después. Alternativa: dejar el sello "pendiente" hasta aprobación del moderador (más fricción, menos fraude). **Asumo: inmediato + auditoría posterior.**
2. **¿Cuáles de los 6 son Asociados?** Propuesta: todos menos Antojitos de la Nutri.
3. **¿Limpieza de imágenes automática o manual?** Propongo manual (botón) + opción "borrar boletas de más de N días". ¿Criterio fijo (ej. 90 días)?
4. **Cooldown auto-servicio:** ¿1 boleta por local cada X horas por usuario, para evitar repetir la misma compra?

6. Orden de ejecución sugerido

Fase 1 → 2 (fundación, sirve para cargar los 6) → Fase 4 + 5 (el flujo de boleta y su auditoría, el corazón) → Fase 3 (superficies visuales). Lo funcional antes que lo cosmético. Cada fase es entregable y verificable por separado.

7. Resumen de archivos

Nuevos: `src/lib/tipoComercio.ts` · `src/app/api/handshake/boleta-scan/route.ts` · `src/app/api/admin/limpiar-boletas/route.ts`

Modificados: `comercios_a_cargar.mjs` , `_ingresar_comercios.mjs` (campo tipo) · `src/app/moderador/page.tsx` (toggle) · `src/app/directorio/page.tsx` , `src/app/descubre/page.tsx` (secciones, badge, filtro) · Home/Destacados · `src/app/canje/page.tsx` (formulario boleta) · `src/app/moderador/sellos/page.tsx` (ver boletas, limpiar) · `src/app/api/admin/anular-sello/route.ts` (borrar imagen al anular) · Reglas de Storage/Firestore (carpeta `boletas/`)